Playwright Interview Questions and Answers – QA Mitra:

1. Elaborate on Playwright's architectural design. How does its "out-of-process" automation model contribute to reliability and speed compared to in-process or WebDriver-based solutions?
2. Discuss the significance of "isolated browser contexts" in Playwright from a test design and execution perspective. How do they enhance test reliability and parallelization?
3. Beyond basic navigation, explain how Playwright handles browser-level events. Provide an example of capturing a new tab/window.
4. Describe Playwright's "auto-waiting" mechanism in detail. How does it contribute to reducing flakiness, and are there scenarios where you might still need explicit waits?
5. What is the role of the Playwright "Test Runner" (@playwright/test)? How does it differ from integrating Playwright with other test frameworks like Jest or Mocha?
6. Discuss Playwright's best practices for choosing locators. Why are "user-facing" locators preferred over traditional CSS/XPath selectors for long-term test stability?
7. When might you still choose to use CSS or XPath selectors in Playwright, despite the recommendations for user-facing locators?
8. Explain the difference between locator.click() and page.click() when multiple matching elements exist. Which is generally safer for robust tests?
9. How do you handle dynamic or non-deterministic elements (e.g., elements with generated IDs) effectively in Playwright?
10. Describe a strategy for interacting with shadow DOM elements in Playwright.
11. How would you structure a Playwright project for a large-scale application to ensure maintainability and scalability? Consider page object model principles.
12. Discuss the various ways Playwright facilitates parallel execution. What are the trade-offs between file-level and project-level parallelism?
13. When would you use test.describe.configure({ mode: 'serial' }), and what are its implications for test execution and reporting?
14. Explain the lifecycle of Playwright fixtures (e.g., page, context, browser). How can you define custom fixtures, and for what use cases would you create them?
15. How would you implement robust authentication handling in a Playwright test suite to avoid re-logging in for every test?

16. Describe how to mock network requests using page.route(). Provide an example of mocking an API response.
17. When would you use page.addScriptTag() or page.addStyleTag()? Provide a practical scenario.
18. How does Playwright's testInfo object assist in reporting or debugging complex test failures?
19. What are "annotations" in Playwright Test, and how can they be used for test management?
20. Describe a scenario where you would use test.slow() and what effect it has.
21. You have a flaky test that occasionally fails without clear reasons. What advanced debugging strategies would you employ in Playwright to diagnose the issue?
22. Explain the benefits of the Playwright Trace Viewer over just using video recordings for debugging.
23. How would you debug a test that's failing only in CI/CD pipelines but passes locally?
24. When debugging, how can you effectively use console.log() statements with Playwright when JavaScript runs in both the Node.js environment and the browser?
25. What strategies would you employ to optimize Playwright test execution speed for a large test suite?
26. How can you measure the performance of specific user flows or page loads within a Playwright test?
27. What is the significance of page.waitForLoadState() and its different options ('load', 'domcontentloaded', 'networkidle')? When would you use each?
28. Describe a typical CI/CD pipeline integration for Playwright tests. What artifacts would you expect to publish?
29. How would you ensure that Playwright tests are executed consistently across different CI environments (e.g., Docker, GitHub Actions, Jenkins)?
30. Beyond the default HTML reporter, how can you customize Playwright's reporting or integrate with third-party reporting tools like Allure?
31. What considerations are important when running Playwright tests in a Dockerized environment?
32. How do Playwright's assertions extend beyond basic equality checks? Give an example of a more advanced assertion.

33. When would you use locator.waitFor() with a state option (e.g., 'detached', 'hidden')? Provide a scenario.
34. How would you gracefully handle expected errors or exceptions during test execution (e.g., an anticipated network error or a specific UI validation message)?
35. Describe how you would implement robust API testing within Playwright, including authentication and response schema validation.
36. What are the principles of writing maintainable and scalable Playwright tests?
37. How do you manage test data effectively in a Playwright test suite for different environments (dev, staging, prod)?
38. Discuss the trade-offs between end-to-end tests and unit/component tests in an overall testing strategy. Where does Playwright fit in?
39. When would you use page.route() to modify requests or responses versus just observing them with page.on('request')?
40. How do you ensure Playwright tests are resilient to changes in UI animations or asynchronous loading patterns?
41. You need to automate a scenario that involves interacting with browser extensions. Is this possible with Playwright? How?
42. How would you test features that rely on geolocation (e.g., location-based services)?
43. How do you handle cookie management (setting, deleting, getting) in Playwright for specific test scenarios?
44. Describe how you would test a website's responsiveness across different devices and screen sizes.
45. What are page.screencast() and page.video() (or configuring video recording)? When is it most effective to capture video?
46. How would you test an application's behavior when offline or with a slow network connection?
47. Explain the page.waitForFunction() method and provide a complex scenario where it would be indispensable.
48. When working with multiple browser contexts, how would you ensure proper cleanup of resources after tests, especially in complex scenarios?
49. How would you approach handling CAPTCHAs or other anti-automation mechanisms in a Playwright test suite for non-production environments?

50. Describe a situation where page.screenshot() with fullPage: true might not capture the full, accurate state of a scrolling page, and what an alternative might be.
51. What is actionability in Playwright? Explain in detail.
52. Describe how you would structure a test automation framework using Playwright/Selenium with JavaScript.
53. How do you handle test data in automation frameworks? Can you show how to load data from external files (e.g., JSON, EXCEL) in your JavaScript tests?
54. What is a Page Object Model (POM)? How have you implemented it in your JavaScript automation projects?
55. How do you ensure your test scripts are maintainable and follow best practices for automation?
56. What strategies do you use to handle dynamic elements in automation scripts written in JavaScript?
57. How would you write an automation script to handle multiple browser contexts or incognito mode in Playwright?
58. What is a test pyramid and how does your JavaScript-based automation framework align with it?
59. How do you handle cross-browser testing in your JavaScript automation framework?
60. How would you use hooks (beforeAll, afterAll, beforeEach, afterEach) in JavaScript testing frameworks like Jest, Mocha, or Playwright to set up and tear down test environments?
61. How do you approach writing tests that involve third-party APIs or microservices in a JavaScript automation framework?

****

Core Architecture & Advanced Concepts

1. **Elaborate on Playwright's architectural design. How does its "out-of-process" automation model contribute to reliability and speed compared to in-process or WebDriver-based solutions?**

Playwright uses a client-server architecture where the test script (client) communicates with a separate, persistent browser process (server) over a WebSocket connection. This out-of-process model means the automation code doesn't directly inject into the browser, preventing crashes if the test code fails, offering greater stability, and enabling faster communication and event handling without the overhead of HTTP requests per command, as seen in WebDriver.

2. **Discuss the significance of "isolated browser contexts" in Playwright from a test design and execution perspective. How do they enhance test reliability and parallelization?**

Isolated browser contexts provide a pristine environment for each test, complete with its own cookies, local storage, session storage, and cache. This isolation prevents tests from leaving behind state that could affect subsequent tests, eliminating flakiness due to shared state. It also enables highly efficient parallelization, as each test runs independently without interference.

3. **Beyond basic navigation, explain how Playwright handles browser-level events. Provide an example of capturing a new tab/window.**

Playwright uses event listeners on Browser, BrowserContext, and Page objects to capture browser-level events. For new tabs/windows. By following the below code we can handle the child window

```
const context = await browser.newContext()

const page = await context.newPage()


await page.goto("url”)

const page1 = context.waitForEvent('page')

await page.locator("[class*='blinking']").click() // - page

const newPage = await page1

await newPage.getByRole('heading', {name:'Documents request'}).waitFor()

const result = await newPage.getByRole('heading', {name:'Documents request'}).isVisible()

expect(result).toBeTruthy()

await page.locator("[name='username']").fill("Testing")

await page.waitForTimeout(5000)
```

OR

```
await page.goto("url")
    const [newPage1] = await Promise.all([
        context.waitForEvent('page'),
        page.locator("[class*='blinking']").click()
    ])
```

4. **Describe Playwright's "auto-waiting" mechanism in detail. How does it contribute to reducing flakiness, and are there scenarios where you might still need explicit waits?**

Playwright's auto-waiting is an intelligent mechanism where action methods (e.g., click(), fill()) and assertions automatically wait for elements to satisfy a set of "actionability checks" (visible, enabled, stable, attached, receives events) before proceeding. This drastically reduces flakiness. Explicit waits might still be needed for non-actionable conditions (e.g., waiting for a specific network request to complete, waiting for a custom animation to fully finish before an action, or waiting for a specific background process that doesn't affect element actionability).

5. **What is the role of the Playwright "Test Runner" (@playwright/test)? How does it differ from integrating Playwright with other test frameworks like Jest or Mocha?**

@playwright/test is Playwright's opinionated, built-in test runner providing fixtures, assertions, parallelization, and reporting out-of-the-box. It's tightly integrated and optimized for Playwright. While Playwright can technically be used with Jest/Mocha by calling Playwright APIs directly, using @playwright/test simplifies setup, offers better performance due to native parallelism, provides powerful built-in debugging tools (Trace Viewer, Inspector), and a more streamlined API for browser interactions.

Advanced Locators & Interactions

6. **Discuss Playwright's best practices for choosing locators. Why are "user-facing" locators preferred over traditional CSS/XPath selectors for long-term test stability?**

User-facing locators (getByText, getByRole, getByLabel, getByTestId) are preferred because they mimic how a human user interacts with the page, making tests more resilient to minor DOM changes. If the text or accessibility role changes, it usually implies a significant UI/UX change that the test should fail for. Traditional CSS/XPath are brittle to DOM refactoring and less readable, leading to higher maintenance.

7. **When might you still choose to use CSS or XPath selectors in Playwright, despite the recommendations for user-facing locators?**

CSS/XPath might be necessary for:

- Elements without semantic roles, text, or explicit test IDs.
- Complex ancestor-descendant relationships not easily expressed otherwise.
- Selecting based on dynamic attributes or specific index within a group.
- Legacy applications where adding test IDs or roles isn't feasible.
- When precise, low-level control over element selection is required.

8. **Explain the difference between locator.click() and page.click() when multiple matching elements exist. Which is generally safer for robust tests?**

page.click() (deprecated for direct use with selectors) would click the first matching element it finds. locator.click() (recommended) operates on a Locator object, which represents a set of potential elements. If a locator matches multiple elements, locator.click() will typically throw an error, explicitly signaling an ambiguity. This behavior of locator.click() is safer as it forces the tester to be precise, preventing unintended interactions.

9. **How do you handle dynamic or non-deterministic elements (e.g., elements with generated IDs) effectively in Playwright?**

This is where robust locator strategies shine. Instead of relying on dynamic IDs, combine user-facing locators with attributes that are more stable:

- getByRole with name (accessible name).
- getByText for static text nearby.
- getByLabel for associated labels.
- getByTestId (custom attribute for testing).
- Chaining locators: page.locator('#parent').getByText('Child Text').
- Using :has() pseudo-class to find an element that contains another element: page.locator('div:has(span.icon-user)').

10. **Describe a strategy for interacting with shadow DOM elements in Playwright.**

Playwright automatically "peers into" Shadow DOM and allows you to use standard locators without any special commands like .shadowRoot. You simply use

page.locator() with CSS selectors, and Playwright will traverse into open shadow roots if the selector matches.

await page.locator('my-component #innerElement').click();

This implicit traversal simplifies interaction significantly.

Test Organization & Advanced Features

11. **How would you structure a Playwright project for a large-scale application to ensure maintainability and scalability? Consider page object model principles.**

For large applications, a Page Object Model (POM) or similar pattern is crucial.

- **Page Object Classes:** Create separate classes (or modules) for each logical page or major component. Each class encapsulates locators and methods for interacting with that part of the UI.
- **Base Page Class:** An optional base class for common methods (e.g., goto, isLoaded).
- **Fixtures for Page Objects:** Use Playwright's fixture system to instantiate and inject page objects into tests.
- **Test Files:** Keep test files clean, importing page objects and focusing on test logic, not element details.
- **Data Driven:** Separate test data from tests.
- **Helper Functions/Utilities:** For common non-page-specific actions.
- **Configuration:** Utilize playwright.config.js for project-wide settings.

12. **Discuss the various ways Playwright facilitates parallel execution. What are the trade-offs between file-level and project-level parallelism?**

Playwright offers parallelization at:

- **Worker Level (Files):** Each test file typically runs in a separate worker process by default.
- **Test Block Level (test.describe.configure({ mode: 'parallel' })):** Tests within a describe block can run in parallel.
- **Project Level:** Using multiple projects in playwright.config.js to run the same tests across different browsers/devices concurrently.

Trade-offs:

- **File-level:** Simple, good for independent tests.
- **Test-block level:** Useful when tests within a file are truly independent but need shared beforeEach/afterEach. Can be tricky if shared state exists.
- Project-level: Best for cross-browser testing, as it provides distinct configurations and results for each.

Maximizing parallelism improves speed but requires robust test isolation to avoid flakiness.

13. **When would you use test.describe.configure({ mode: 'serial' }), and what are its implications for test execution and reporting?**

mode: 'serial' is used when tests within a describe block have strict dependencies on each other and must execute sequentially. If any test in a serial block fails, all subsequent tests in that same block are skipped. This is useful for multi-step workflows where a failure at an early step makes later steps irrelevant, helping to prevent cascading failures and provide clearer reports.

14. **Explain the lifecycle of Playwright fixtures (e.g., page, context, browser). How can you define custom fixtures, and for what use cases would you create them?**

Playwright fixtures provide resources for tests and are managed by the test runner.

- page: Per-test, isolated Page object.
- context: Per-test, isolated BrowserContext (contains one or more pages).
- browser: Per-worker, shared Browser instance.

Custom fixtures are defined using test.extend().

Scenarios:

- **Authentication:** A fixture that logs in and provides an authenticated page or context.
- **Database Setup/Teardown:** A fixture that prepares test data in a database.
- **API Client:** A fixture providing a pre-configured API client object.
- **Shared Page Objects:** A fixture that instantiates and injects a PageObject instance.

- **Specialized Test Environment:** Setting up unique browser capabilities for a subset of tests.

15. **How would you implement robust authentication handling in a Playwright test suite to avoid re-logging in for every test?**

The most robust way is to save and reuse authentication state:

1. **Global Setup (globalSetup):** Log in once, capture the storageState (cookies, local storage, session storage) of the authenticated session using await context.storageState({ path: 'auth.json' });.
2. playwright.config.js: Configure projects to use: { storageState: 'auth.json' }.

Each subsequent test will automatically start with the pre-authenticated session. This significantly speeds up tests by avoiding repeated login flows.

16. **Describe how to mock network requests using page.route(). Provide an example of mocking an API response.**

page.route() intercepts network requests based on a URL pattern and allows you to fulfill them with custom data.

```
await page.route('**/api/users', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([{ id: 1, name: 'Mock User' }]),
  });
});
// Now, any request to /api/users will receive the mocked response
await page.goto('/dashboard'); // Dashboard makes API call
await expect(page.getByText('Mock User')).toBeVisible();
```

17. **When would you use page.addScriptTag() or page.addStyleTag()? Provide a practical scenario.**

These methods inject JavaScript or CSS into the page, respectively.

Practical Scenario:

- **Test Data Setup:** Injecting a script to populate a client-side store (e.g., Redux, Vuex) with specific test data, bypassing UI interaction.

- **Debugging/Profiling:** Injecting a custom JavaScript profiler or logging utility.
- **UI Tweaks:** Injecting CSS to hide irrelevant elements or highlight elements under test during debugging.
- **Stubbing Client-Side Functions:** Overriding a browser's native function (e.g., Date.now()) for time-sensitive tests.

18. **How does Playwright's testInfo object assist in reporting or debugging complex test failures?**

The testInfo object provides runtime context about the current test (testInfo.title, testInfo.status, testInfo.error, testInfo.retry, testInfo.outputDir). It's crucial for:

- **Dynamic Screenshots:** Saving screenshots with test-specific names in testInfo.outputDir.
- **Conditional Logging:** Logging extra details only if testInfo.status === 'failed'.
- **Error Handling:** Accessing testInfo.error to customize error messages or integrate with external error reporting.
- **Artifact Management:** Storing test-specific data in the dedicated output directory.

19. **What are "annotations" in Playwright Test, and how can they be used for test management?**

Annotations (test.skip(), test.only(), test.fixme(), test.fail()) provide metadata about a test's expected behavior or state.

- test.skip(): Skips the test.
- test.only(): Runs only this test (and other only tests).
- test.fixme(): Marks a flaky/broken test that should be fixed, but runs it. If it fails, it's considered skipped (not a failure). If it passes, it's flaky.
- test.fail(): Marks a test as expected to fail. If it fails, it's considered passed. If it passes, it's failed (unexpected success).

They help in managing test suites, focusing on specific tests, and distinguishing between known issues and new regressions.

20. **Describe a scenario where you would use test.slow() and what effect it has.**

test.slow() triples the default timeout for a specific test. You would use it for tests that are inherently lengthy due to complex interactions, large data processing, or external

API calls that genuinely take a long time to respond. It helps avoid premature timeouts for valid, long-running tests without globally increasing timeouts.

Debugging & Troubleshooting

21. **You have a flaky test that occasionally fails without clear reasons. What advanced debugging strategies would you employ in Playwright to diagnose the issue?**

    1. **Trace Viewer:** The absolute first step. Run with --trace on and analyze the trace. Look for missing actions, unexpected DOM changes, network activity, or console errors at the point of failure.
    2. **page.pause():** Insert await page.pause() at strategic points to interact with the browser manually in Playwright Inspector and observe the state.
    3. **Video Recording:** Configure video recording (e.g., video: 'on-first-retry') to visually review the exact moment of failure.
    4. **Network Interception:** Use page.route() to mock problematic network calls or page.on('request')/'response' to log all network traffic for anomalies.
    5. **Console Logs:** Check page.on('console') for client-side errors or warnings.
    6. **Retries:** Use --retries to observe flakiness patterns and gather more trace/video data.
    7. **Run in Headed Mode:** --headed can help visualize subtle UI issues.

22. **Explain the benefits of the Playwright Trace Viewer over just using video recordings for debugging.**

Trace Viewer offers a much richer debugging experience than just video:

- **Step-by-step Execution:** See each Playwright action, with pre- and post-action DOM snapshots and selectors.
- **DOM Snapshots:** Inspect the DOM at any point, including CSS and computed styles.
- **Network Logs:** View all network requests and responses, their timings, and content.
- **Console Logs:** Access console.log, warn, error messages from the browser.
- **Source Code Navigation:** See the exact line of your test code corresponding to each action.

- **Performance Metrics:** View CPU and memory usage.
- Searchability: Easily search through logs and events.

Video only shows the visual outcome; Trace provides the underlying technical details.

23. **How would you debug a test that's failing only in CI/CD pipelines but passes locally?**
    1. **Trace on CI:** Configure CI to capture traces (--trace on) and download them for local analysis using npx playwright show-trace. This is critical.
    2. **Video on CI:** Capture videos on retry or failure.
    3. **Console/Network Logs:** Ensure CI logs all browser console messages and network activity.
    4. **Environment Parity:** Check for environmental differences: browser versions, operating system, display resolution, network latency, system resources (CPU, RAM).
    5. **Headless vs. Headed:** CI often runs headless; try running locally headless to reproduce.
    6. **Timeouts:** CI environments can be slower; review and potentially increase relevant timeouts if necessary.
    7. **Artifacts:** Ensure screenshots and other failure artifacts are saved and accessible from CI.
24. **When debugging, how can you effectively use console.log() statements with Playwright when JavaScript runs in both the Node.js environment and the browser?**
    - console.log() in your test file (Node.js) logs to your terminal.
    - page.on('console', msg => console.log(msg.text())) allows you to capture and print console.log() messages *from the browser's context* to your terminal. This is vital for debugging client-side JavaScript issues.

Performance & Optimization

25. **What strategies would you employ to optimize Playwright test execution speed for a large test suite?**
    - **Maximize Parallelism:** Configure fullyParallel: true and set workers appropriately (e.g., workers: '50%' of CPU cores).
    - **Authentication State Reuse:** Use storageState via globalSetup to avoid repeated logins.

- **Network Mocking/Interception:** Stub external API calls or third-party scripts that aren't critical for the test to reduce network latency.
- **Headless Mode:** Always run tests in headless mode in CI/CD.
- **Reduce Redundancy:** Refactor common setup into fixtures or beforeEach/beforeAll.
- **Targeted Testing:** Use test.only, test.skip, or --grep to run only necessary tests during development.
- **Efficient Locators:** Use Playwright's recommended user-facing locators, which are often more performant than complex CSS/XPath.
- **Avoid Unnecessary Waits:** Rely on auto-waiting. Only use explicit waits when genuinely necessary.

26. **How can you measure the performance of specific user flows or page loads within a Playwright test?**
    - **Performance API:** Use page.evaluate(() => performance.timing) or performance.getEntriesByType('paint') to access browser's Performance API.
    - **Navigation Timing:** Log page.metrics() to get various browser performance metrics.
    - **Network Metrics:** Analyze request and response events to measure individual network call durations.
    - **page.waitForLoadState():** Use 'networkidle' for more accurate "fully loaded" state.
    - **Custom Timers:** Use Date.now() or performance.now() in your Node.js code to time specific actions or blocks.
    - **Trace Viewer:** Provides detailed network and rendering performance insights.
27. **What is the significance of page.waitForLoadState() and its different options ('load', 'domcontentloaded', 'networkidle')? When would you use each?**

page.waitForLoadState() waits for specific page loading events.

- 'load': Waits until the load event fires (all resources including images, stylesheets are loaded). Good for general page readiness.
- 'domcontentloaded': Waits until the DOM is parsed and constructed. Useful when you need to interact with elements but don't need all media to be loaded. Faster than 'load'.

- 'networkidle': Waits until there are no more than 0 or 1 network connections for at least 500ms. This is the most robust for dynamically loaded content and often indicates the page is truly "settled." Use for complex SPAs.

CI/CD Integration & Reporting

28. **Describe a typical CI/CD pipeline integration for Playwright tests. What artifacts would you expect to publish?**

A typical CI/CD pipeline would involve:

1. **Checkout Code:** Get the latest version of the test suite.
2. **Install Dependencies:** npm install (or equivalent for other languages).
3. **Install Playwright Browsers:** npx playwright install --with-deps.
4. **Run Tests:** npx playwright test --reporter=junit,html --output=test-results --workers=X.
5. **Publish Artifacts:** Upload test-results/ directory containing:
   - HTML report (for visual review).
   - JUnit XML report (for CI dashboard integration).
   - Screenshots and videos (for failed tests).
   - Traces (for deep debugging of failures).
6. **Report Status:** Update build status based on test results.

29. **How would you ensure that Playwright tests are executed consistently across different CI environments (e.g., Docker, GitHub Actions, Jenkins)?**

- **Docker Containers:** Package your test environment (Node.js, Playwright, browser binaries) into a Docker image. This provides maximum consistency.
- **Consistent Node.js/Python/Java Versions:** Use .nvmrc, pyenv, or specific language version managers.
- **Playwright Browser Installation:** Always use npx playwright install --with-deps in the CI pipeline to ensure correct browser binaries for the environment.
- **Environment Variables:** Use environment variables for sensitive data or dynamic configurations (e.g., BASE_URL).
- **Headless Mode:** Ensure tests run headless in CI unless a VNC server is set up.

30. **Beyond the default HTML reporter, how can you customize Playwright's reporting or integrate with third-party reporting tools like Allure?**
    - **Custom Reporters:** Implement a custom reporter by extending Playwright's Reporter class, allowing full control over output.
    - **Multiple Reporters:** Configure multiple built-in reporters (e.g., ['list'], ['html'], ['json']).
    - **Third-Party Plugins:** Use community-contributed plugins (e.g., playwright-test-allure-reporter). These are installed separately and configured in playwright.config.js.
31. **What considerations are important when running Playwright tests in a Dockerized environment?**
    - **Image Size:** Use minimal base images (e.g., node:lts-slim).
    - **Browser Dependencies:** Ensure all browser runtime dependencies are installed (npx playwright install --with-deps). Often, a pre-built Playwright Docker image is best.
    - **Headless Execution:** Playwright runs headless by default in Docker.
    - **Memory/CPU Limits:** Allocate sufficient resources to the container.
    - **Artifact Volume Mounting:** Mount volumes to persist test reports, screenshots, and videos outside the container.
    - **Network Configuration:** Ensure the container can reach your application under test.

Advanced Assertions & Error Handling

32. **How do Playwright's assertions extend beyond basic equality checks? Give an example of a more advanced assertion.**

Playwright's assertions are built on the Jest expect library but extended with specific web-first assertions that auto-wait for conditions.

Advanced Example:

- await expect(locator).toBeEnabled(): Waits for the element to become enabled.
- await expect(locator).toHaveAttribute('data-status', 'active'): Waits for an attribute to have a specific value.
- await expect(page.url()).toContain('dashboard'): Checks part of the URL.

- await expect(page).toHaveScreenshot({ maxDiffPixels: 100 }): Visual comparison with pixel tolerance.

33. **When would you use locator.waitFor() with a state option (e.g., 'detached', 'hidden')? Provide a scenario.**

You use locator.waitFor() to explicitly wait for an element to reach a specific state that's not automatically covered by actionability checks.

Scenario: Waiting for a loading spinner to disappear (state: 'detached' or state: 'hidden') before asserting content on the page.

```
const loadingSpinner = page.locator('#spinner');

await expect(loadingSpinner).toBeHidden(); // Auto-waits

// OR

await loadingSpinner.waitFor({ state: 'detached' }); // Explicitly waits for it to be removed
```

34. **How would you gracefully handle expected errors or exceptions during test execution (e.g., an anticipated network error or a specific UI validation message)?**
    - **try...catch blocks:** For handling synchronous or asynchronous code where an error is anticipated, allowing you to assert on the error message or type.
    - **page.on('dialog'):** For browser dialogs.
    - **page.on('pageerror'):** To catch uncaught exceptions on the client side.
    - **page.waitForEvent('requestfailed'):** To specifically wait for and assert on a failed network request.
    - **test.fail() annotation:** To explicitly mark a test that is expected to fail due to a known bug.
35. **Describe how you would implement robust API testing within Playwright, including authentication and response schema validation.**
    1. **Request Context:** Use the request fixture, which provides a clean API for HTTP requests.
    2. **Authentication:** Pass authentication headers (e.g., Bearer token) directly in the request options, or reuse authenticated state from UI login if using storageState.
    3. **Request Building:** Use request.post(), request.get(), request.put(), request.delete() with data and headers options.

4. **Status Code Validation:** expect(response.status()).toBe(200); or expect(response.ok()).toBeTruthy();.
5. **Response Body Validation:**
    - const json = await response.json();
    - Deep equality: expect(json).toEqual({ key: 'value' });
    - Partial matching: expect(json).toMatchObject({ key: 'value' });
    - **Schema Validation:** Integrate a JSON schema validator (e.g., ajv or zod) into your test. Fetch the schema, then validate the API response against it.

Best Practices & Maintainability

36. **What are the principles of writing maintainable and scalable Playwright tests?**
    - **Readability:** Clear test names, well-commented code, logical flow.
    - **Modularity:** Use Page Object Model, custom fixtures, and helper functions.
    - **Reusability:** Abstract common actions into methods, leverage fixtures.
    - **Isolation:** Ensure tests are independent and don't rely on previous test state.
    - **Resilience:** Use robust, user-facing locators.
    - **Efficiency:** Optimize for speed (parallelism, auth reuse).
    - **Reporting:** Generate clear, actionable reports.
    - **Version Control:** Standard Git practices.
37. **How do you manage test data effectively in a Playwright test suite for different environments (dev, staging, prod)?**
    - **Configuration Files:** Use environment-specific .env files or separate configuration modules for URLs, credentials.
    - **Data Factories/Generators:** Create functions/classes to generate realistic but predictable test data (e.g., faker.js).
    - **API for Setup:** Use API calls (via request fixture) in beforeAll or setup fixtures to create test data directly in the database/backend, bypassing UI.
    - **Database Seeding:** Integrate with a database seeding tool for more complex data needs.

- **Parameterization:** Use external data files (CSV, JSON) for data-driven tests.

38. **Discuss the trade-offs between end-to-end tests and unit/component tests in an overall testing strategy. Where does Playwright fit in?**
    - **Unit/Component Tests:** Fast, isolated, test small units of code. Catch bugs early. Playwright doesn't fit directly here.
    - **End-to-End Tests (E2E):** Slow, cover full user flows, interact with UI and backend. Verify system integration. Playwright excels here.

Trade-offs: E2E tests are expensive to write and maintain, and slow to run. Unit/component tests provide faster feedback but miss integration issues.

Playwright's Fit: Playwright is ideal for the E2E layer, acting as a final confidence check for critical user journeys. It can also bridge to component testing by interacting with isolated components rendered in a browser. A balanced strategy uses fewer, high-value E2E tests and more unit/component tests.

39. **When would you use page.route() to modify requests or responses versus just observing them with page.on('request')?**
    - **page.on('request'):** Use when you only need to observe, log, or assert on network requests without changing them (e.g., ensuring a specific API call was made).
    - **page.route():** Use when you need to *alter* the network behavior:
        - **Mocking:** Providing fake responses for external APIs or unstable services.
        - **Blocking:** Preventing certain requests (e.g., analytics, ads).
        - **Modifying:** Changing request headers, post data, or response bodies on the fly.
        - **Simulating Errors:** Returning specific HTTP error codes for testing error handling.
        - **Performance Testing:** Blocking slow resources.

40. **How do you ensure Playwright tests are resilient to changes in UI animations or asynchronous loading patterns?**
    - **Rely on Auto-Waiting:** Playwright's actionability checks already wait for elements to be stable.
    - **page.waitForLoadState('networkidle'):** For pages with many dynamic requests.

- **locator.waitFor({ state: 'detached'/'hidden' }):** To explicitly wait for loading indicators to disappear.
- **page.waitForFunction():** For highly custom, JavaScript-driven conditions that aren't tied to DOM element states (e.g., waiting for a specific variable to be set, or a custom event to fire).
- **Assertions with Auto-Retry:** Playwright's expect assertions auto-retry, handling transient states.

Problem Solving & Advanced Scenarios

41. **You need to automate a scenario that involves interacting with browser extensions. Is this possible with Playwright? How?**

Yes, Playwright supports loading browser extensions. You launch the browser with the args option to specify the extension directory:

- For Chromium: --disable-extensions-except=<path/to/extension>, --load-extension=<path/to/extension>

Once loaded, you can get a reference to the extension's background page or popup page by finding the Page object with its specific URL or title, then interact with it like any other page.

42. **How would you test features that rely on geolocation (e.g., location-based services)?**

Playwright's context.setGeolocation() method allows you to set a precise latitude and longitude for the browser context. You also need to grant geolocation permissions:

```
const context = await browser.newContext({
  permissions: ['geolocation'],
  geolocation: { latitude: 34.052235, longitude: -118.243683 },
});
const page = await context.newPage();
await page.goto('https://maps.google.com'); // Or your app
// Now the page will use the mocked geolocation
```

43. **How do you handle cookie management (setting, deleting, getting) in Playwright for specific test scenarios?**

You use methods on the BrowserContext object:

- **context.addCookies():** To set specific cookies.
- **context.cookies():** To retrieve all cookies for the context.

- **context.clearCookies():** To delete all cookies.
- **context.storageState():** To save/load all cookies, local storage, and session storage.

44. **Describe how you would test a website's responsiveness across different devices and screen sizes.**
    - **playwright.config.js projects:** Define multiple projects, each configured with different viewport dimensions or using ...devices from @playwright/test for pre-defined mobile/tablet settings.
    - **page.setViewportSize():** Change the viewport size dynamically within a test for specific assertions.
    - **Visual Regression Testing:** Combine with toHaveScreenshot() to catch layout shifts or rendering issues on different viewports.
45. **What are page.screencast() and page.video() (or configuring video recording)? When is it most effective to capture video?**

These refer to Playwright's ability to record a video of the test execution.

page.video() or video option in config captures a video of each test.

When effective:

- **Debugging Flaky Tests:** Visual evidence of transient failures.
- **Showcasing Bugs:** Easily demonstrate reproduction steps.
- **Non-Reproducible Bugs:** Capture rare scenarios.
- Complex UI Interactions: Verify subtle animations or flows.

It's most effective when set to capture on first retry (video: 'on-first-retry') or on failure (video: 'on-failure') to save disk space while still getting valuable debugging artifacts.

46. **How would you test an application's behavior when offline or with a slow network connection?**

You can use page.route() to intercept requests and simulate network conditions:

- **Offline:** route.abort() or route.fulfill({ status: 503 }) for all requests.
- **Slow Network:** route.fulfill({ response: actualResponse, delay: 2000 }) to add artificial latency.
- **Throttling:** Use context.emulateNetworkConditions() to set latency, download/upload throughput.

await context.emulateNetworkConditions({

offline: false,

latency: 100, // 100ms latency

*downloadThroughput: 1000 * 1024, // 1 Mbps*

*uploadThroughput: 500 * 1024, // 0.5 Mbps*

});

47. **Explain the page.waitForFunction() method and provide a complex scenario where it would be indispensable.**

page.waitForFunction() waits for a JavaScript function inside the browser's context to return a truthy value. It constantly polls the function until it returns true or times out.

Indispensable Scenario: Waiting for a complex, non-DOM related condition to be met by client-side logic, such as:

- Waiting for a specific global variable to be set by a third-party script.
- Waiting for a Redux/Vuex store to reach a certain state after an asynchronous operation.
- Waiting for a custom animation library to report completion via its internal state.
- Waiting for an event listener that doesn't affect DOM elements to fire.

await page.waitForFunction(() => window.myGlobalState === 'ready', { timeout: 10000 });

48. **When working with multiple browser contexts, how would you ensure proper cleanup of resources after tests, especially in complex scenarios?**
    - **Fixtures:** Use custom context or page fixtures that handle cleanup in their tear down phase (e.g., async ({ context }, use) => { await use(context); await context.close(); }).
    - **afterEach / afterAll hooks:** Ensure any manually created contexts or pages are closed within these hooks.
    - **Global Teardown:** For resources shared across the entire run (e.g., a server started by globalSetup), use globalTeardown to shut them down.
    - **Test Isolation:** Adhere strictly to test isolation principles; if tests are truly isolated, Playwright's built-in cleanup handles most cases.
49. **How would you approach handling CAPTCHAs or other anti-automation mechanisms in a Playwright test suite for non-production environments?**

For non-production (QA, Dev) environments:

- **Disable CAPTCHA:** Request development teams to provide a bypass (e.g., specific header, API endpoint to solve programmatically, or an environment variable to disable it). This is the best approach for stable automation.
- **Mock API:** If CAPTCHA uses an API, mock its response using page.route() to always return a "solved" status.
- **Test-specific bypass:** Inject JavaScript via page.evaluate() to disable the CAPTCHA script or set a solved state.
- **For production/real-world (less ideal for automation):** Integration with CAPTCHA solving services (e.g., 2Captcha), but this adds cost, latency, and fragility.

50. Describe a situation where page.screenshot() with fullPage: true might not capture the full, accurate state of a scrolling page, and what an alternative might be.

fullPage: true attempts to scroll the page and stitch screenshots. However, it might not be fully accurate if:

- **Sticky/Fixed Elements:** Elements with position: fixed or sticky might appear multiple times or be misplaced.
- **Lazy Loading:** Content that only loads when scrolled into view might not fully render if the page is scrolled too quickly by Playwright's internal mechanism, or if the networkidle state isn't long enough.
- **Complex Layouts:** Pages with custom scrollable containers within the main body.

Alternative:

- **Manual Scrolling & Screenshots:** For highly problematic pages, manually scroll through sections (page.evaluate() to control window.scrollTo or element scrollTop), take multiple screenshots, and then programmatically stitch them together (outside of Playwright, using an image manipulation library).
- **Component-level screenshots:** Screenshot individual, self-contained components rather than the entire page.
- **Visual testing tools with specific "full page" handling:** Some dedicated visual regression tools might have more sophisticated internal mechanisms for this.

51. What is actionability in Playwright? Explain in detail

- Playwright architecture has a special type of checks before performing any actions on elements. The checks are called actionability checks.
- For example when you do click operation page.click()
- It will perform many checks internally such as
- Element is attached to DOM
- Element is Visible
- Element is Stable and animation is completed(if any)
- Element is ready to receive the events
- Element is enabled.

This mechanism is also called automatic waiting in the Playwright. Since the Playwright performs all the above checks by default one is no need to perform the above checks manually.